Technical Interview Challenge: Task Creation

Table of Contents

1. Overview
2. Prerequisites & Setup
3. Your Assignment
4. Understanding the Task System
5. Required Files
6. The Task: JSON Schema Validator
7. Validation Requirements
8. Error Handling Requirements
9. Output Format Specification
10. Grader Requirements
11. Solution Requirements
12. Testing Your Implementation
13. Submission
14. Troubleshooting

Overview

We're building tasks that evaluate & test LLM-based agents across multiple domains including:

- Data Science and Analysis
- System Administration
- Networking and Security
- Software Development
- DevOps and Infrastructure

Your challenge is to create a **complete task** from scratch that can be used to test an AI agent's problem-solving capabilities.

Prerequisites & Setup

Before you begin, ensure you have the required tools installed.

Required Dependencies

Dependency	Version	Installation
Python	3.8+	python.org
Docker Desktop	Latest	docker.com

Optional but recommended Dependencies

Dependency	Purpose	Installation
PyYAML	Show task difficulty in `list-tasks`	`pip install pyyaml`

Setup Steps

1. Make the CLI executable

```
chmod +x apex-arena
```

2. Docker Desktop WSL Integration (Windows only)

If you're using WSL on Windows:

1. Open **Docker Desktop**
2. Go to **Settings** → **Resources** → **WSL Integration**
3. Enable integration for your WSL distro (e.g., Ubuntu)
4. Click **Apply & Restart**
5. In WSL terminal, verify with:

```
docker context use desktop-linux
docker ps
```

Your Assignment

You will be given the **input data only**. Your job is to create a complete, production-ready task that includes all required components.

What You Must Deliver

A **zipped folder** named `schema_validator.zip` containing:

```
schema_validator/
├── data/
│   └── validation_request.json    # (provided by us - copy it here)
├── tests/
│   └── validation_request.json    # (copy of data for grader security)
├── Dockerfile                      # Container environment setup
├── task.yaml                       # Problem specification for the agent
├── grader.py                       # Evaluation logic (YOU WRITE THIS)
└── solution.sh                     # Reference solution (YOU WRITE THIS)
```

Understanding the Task System

How Agent Evaluation Works

When we evaluate an AI agent, the following happens:

1. A **Docker container** is built from the `Dockerfile` - this creates an isolated environment
2. The agent receives the **prompt** from `task.yaml` - this describes the problem to solve
3. The agent works inside the container, reading input files from `/workdir/data/`
4. The agent writes its solution to the specified output location
5. The **grader** (`grader.py`) runs to evaluate the agent's work

6. A score (0.0 or 1.0) is returned based on correctness

Security Model

- `/workdir/data/` - Accessible to the agent (input files)
- `/tests/` - **NOT accessible** to the agent (root only) - contains grader resources
- The grader runs with root privileges and can access `/tests/`
- This prevents the agent from cheating by reading expected answers

---

Required Files

Dockerfile

Use this as your starting point:

```
FROM python:3.11-slim

# Install bash (required for solution.sh)
RUN apt-get update && apt-get install -y --no-install-recommends \
    bash \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# Create model user and workdir
RUN useradd -m -s /bin/bash model \
    && mkdir -p /workdir \
    && chown model:model /workdir

WORKDIR /workdir

# No additional Python dependencies needed - json is built-in

# ──────────── DO NOT CHANGE BELOW ─────────────

COPY ./tests/ /tests/
COPY ./solution.sh /tests/
COPY ./grader.py /tests/
COPY ./data /workdir/data

RUN chown -R model:model /workdir/data \
    && chmod -R 700 /workdir/data
```

Important Notes:

- The base image `python:3.11-slim` provides Python 3.11 with a minimal footprint
- Never modify the "DO NOT CHANGE BELOW" section
- Consider not installing libraries that would trivialize the task (e.g., `jsonschema`)

task.yaml

Use this structure for your task specification:

```
prompt: |
  [Write a clear, detailed prompt here that explains:]

  1. What problem the agent needs to solve
  2. Where input files are located
  3. Where to write output files
  4. Exact output format expected
  5. All validation rules and requirements
  6. Allowed libraries/tools

  Be SPECIFIC and UNAMBIGUOUS - the agent only knows what you tell it.

metadata:
  difficulty: hard
  category: data_validation
  required_tools:
    - bash
    - python
  tags:
    - json
    - schema
    - validation
  time_limit: 600
  memory_limit: 512
  max_agent_timeout_sec: 400
  expert_time_estimate_min: 25
  junior_time_estimate_min: 50
```

The Task: JSON Schema Validator

You must create a task that evaluates an agent's ability to build a JSON Schema validator.

Input Data (Provided)

We provide `validation_request.json` containing:

- Multiple JSON Schema definitions
- Multiple documents to validate against those schemas

What the Agent Must Do

1. Read the input file
2. Validate each document against its corresponding schema
3. Report all validation errors with detailed information
4. Write structured results to an output file

What YOU Must Implement

1. **`task.yaml`** - Write a comprehensive prompt that specifies:
   - All validation rules (types, constraints, formats)
   - Exact output format
   - Error reporting requirements
   - Edge cases to handle
2. **`solution.sh`** - A reference solution that:
   - Correctly implements ALL validation rules
   - Handles ALL edge cases
   - Produces perfect output scoring 1.0
3. **`grader.py`** - Evaluation logic that:
   - Computes ground truth independently

- Compares agent output against expected results
- Returns binary score (1.0 or 0.0)
- Provides informative feedback on failures

Validation Requirements to Cover

Your implementation MUST handle all of these:

1. Type Validation

Type	Requirements
`string`	Must be a string
`number`	Any numeric value (int or float)
`integer`	Whole numbers only (no decimals)
`boolean`	true or false
`null`	null value
`array`	Must be an array
`object`	Must be an object

2. String Constraints

Constraint	Validation Rule
`minLength`	String length >= value
`maxLength`	String length <= value
`pattern`	Must match regex pattern
`format: email`	Valid email format
`format: date`	YYYY-MM-DD format
`format: uuid`	8-4-4-4-12 hex format
`format: uri`	http:// or https:// URL

3. Number Constraints

Constraint	Validation Rule
`minimum`	Value >= minimum
`maximum`	Value <= maximum
`exclusiveMinimum`	When true: value > minimum
`exclusiveMaximum`	When true: value < maximum
`multipleOf`	Value divisible by this number

> **NOTE:** This uses JSON Schema Draft 4 style where `exclusiveMinimum` and `exclusiveMaximum` are boolean modifiers.

4. Array Constraints

Constraint	Validation Rule
`minItems`	Array length >= value
`maxItems`	Array length <= value
`uniqueItems`	All elements must be unique
`items`	Each element must match schema

5. Object Constraints

Constraint	Validation Rule
`required`	Listed properties must exist
`properties`	Property schemas to validate
`additionalProperties`	When false: no extra properties allowed

6. Enum Constraint

Constraint	Validation Rule
`enum`	Value must be in the list

Error Handling Requirements

Your grader and solution must handle ALL possible errors:

File/Input Errors

- Input file not found
- Invalid JSON syntax
- Missing required fields in input structure

- Schema ID referenced but not defined

Validation Errors (to report per document)

- Type mismatch
- Missing required property
- String too short / too long
- Pattern mismatch
- Invalid format (email, date, uuid, uri)
- Number out of range (min/max violations)
- Exclusive boundary violations
- Array too short / too long
- Duplicate items in array
- Invalid array item type
- Additional properties not allowed
- Enum value not in list

Error Output Format

Each error must include:

```
{
  "path": "$.fieldName",
  "message": "Human-readable description",
  "constraint": "constraintName",
  "expected": "what was expected",
  "actual": "what was found"
}
```

Output Format Specification

The agent must produce this exact structure:

```json
{
  "validation_results": [
    {
      "document_id": "doc_001",
      "schema_id": "schema_001",
      "valid": true,
      "errors": []
    },
    {
      "document_id": "doc_002",
      "schema_id": "schema_001",
      "valid": false,
      "errors": [
        {
          "path": "$.email",
          "message": "Invalid email format",
          "constraint": "format",
          "expected": "email",
          "actual": "not-an-email"
        }
      ]
    }
  ],
  "summary": {
    "total_documents": 10,
    "valid_documents": 3,
    "invalid_documents": 7,
    "total_errors": 19,
    "errors_by_constraint": {
      "type": 5,
      "required": 3,
      "format": 4,
      "minimum": 2,
      "pattern": 2,
      "enum": 1,
      "uniqueItems": 1,
      "additionalProperties": 1
    }
  }
}
```

Grader Requirements

Your `grader.py` must:

1. Load Data Securely

```
# Always try /tests/ first (agent cannot access this)
DATA_FILE_TESTS = Path("/tests/validation_request.json")
DATA_FILE_WORKDIR = Path("/workdir/data/validation_request.json")

for path in [DATA_FILE_TESTS, DATA_FILE_WORKDIR]:
    if path.exists():
        data = json.loads(path.read_text())
        break
```

2. Compute Ground Truth

- Implement the SAME validation logic as solution.sh
- Never hardcode expected answers
- Calculate expected results dynamically from input

3. Compare Results

- Load agent's output from `/workdir/validation_results.json`
- Compare against computed ground truth
- Check: document validity, error counts, error details, summary stats

4. Return GradingResult

```
from apex_arena._types import GradingResult

def grade(_: str) -> GradingResult:
    # ... validation logic ...

    return GradingResult(
        score=1.0 if all_correct else 0.0,
        subscores={"validation": score},
        weights={"validation": 1.0},
        feedback="Detailed feedback message"
    )
```

5. Handle Edge Cases

- Missing output file → score 0.0 with feedback
- Invalid JSON in output → score 0.0 with feedback
- Missing required fields → score 0.0 with feedback
- Provide CLEAR feedback explaining what went wrong

Solution Requirements

Your `solution.sh` must:

1. Be a bash script that runs Python code
2. Read from `/workdir/data/validation_request.json`
3. Write to `/workdir/validation_results.json`
4. Use ONLY Python standard library (`json`, `re`, `math`)
5. Implement ALL validation rules correctly
6. Handle ALL edge cases
7. Score 1.0 when evaluated by your grader

Template

```
#!/usr/bin/env bash
set -euo pipefail

python3 << 'PY'
import json
import re
from pathlib import Path
from typing import Dict, Any, List

def main():
    # Load input
    data = json.loads(Path("/workdir/data/validation_request.json").read_text())

    # Your validation logic here
    results = validate_all(data)

    # Write output
    Path("/workdir/validation_results.json").write_text(
        json.dumps(results, indent=2)
    )

if __name__ == "__main__":
    main()
PY
```

Testing Your Implementation

Before submitting, you **MUST** test your task using the Apex Arena CLI.

Available CLI Commands

Validate Task Structure

```
./apex-arena validate-task schema_validator
```

This checks that your task has the correct file structure. Checks performed:

Check	Type	Description
Required files exist	Error	`Dockerfile`, `grader.py`, `solution.sh`, `task.yaml`
Required directories exist	Error	`tests/` with at least one file
Dockerfile uses base image	Error	Must have `FROM python:3.11-slim`
`grader.py` has grade function	Error	Must define `def grade(`
`task.yaml` has prompt	Error	Must have `prompt` field
Misplaced files	Error	e.g., `validation.json` in root instead of `tests/`
Dockerfile copies files	Warning	Should copy to `/tests/`
`solution.sh` has shebang	Warning	Should start with `#!/bin/bash`
`task.yaml` has metadata	Warning	Should include difficulty

Test Your Solution

```
./apex-arena test-solution schema_validator
```

Runs your `solution.sh` in a Docker container and grades the result.

Options:

Flag	Description
`--force`, `-f`	Force rebuild the Docker image
`--verbose`, `-v`	Show full output from solution
`--keep-container`, `-k`	Keep container running after test (for debugging)

Examples:

```
# Basic test
./apex-arena test-solution schema_validator

# Force rebuild the Docker image
./apex-arena test-solution schema_validator --force

# Show full output and keep container for debugging
./apex-arena test-solution schema_validator --verbose --keep-container
```

List Available Tasks

```
./apex-arena list-tasks
```

Shows all tasks in the `tasks/` directory with their difficulty level.

Testing Checklist

Before submitting, verify:

1. **Task validates successfully:**

   ```
   ./apex-arena validate-task schema_validator
   # Should show no errors
   ```

2. **Solution achieves score 1.0:**

   ```
   ./apex-arena test-solution schema_validator
   # Must show: Score: 1.0
   ```

3. **Grader returns 0.0 for broken solutions:**
   - Test with missing output file
   - Test with malformed JSON
   - Test with incorrect validation results

Submission

Folder Structure

```
schema_validator/
├── data/
│   └── validation_request.json
├── tests/
│   └── validation_request.json
├── Dockerfile
├── task.yaml
├── grader.py
└── solution.sh
```

How to Submit

1. Create the folder structure above
2. Copy the provided `validation_request.json` to both `data/` and `tests/`
3. Implement all required files
4. **Test your solution using the CLI:**

   ```
   ./apex-arena validate-task schema_validator
   ./apex-arena test-solution schema_validator
   ```

5. **Take a screenshot** of the CLI output showing:
   - The `test-solution` command executed
   - The final score of **1.0**
   - Any relevant feedback/output
   - The `validate-task` command output
6. Zip the folder: `zip -r schema_validator.zip schema_validator/`
7. **Submit both:**
   - The `schema_validator.zip` file
   - The screenshot proving your solution scores 1.0
   - The screenshot proving your task is valid

> **IMPORTANT:** Your submission will not be considered complete without the screenshots showing a score of 1.0 from `test-solution` and a passing `validate-task` output.

Provided Input Data

The file `validation_request.json` is provided separately. It contains:

- **3 schemas**: User, Product, Order
- **10 documents**: Mix of valid and invalid
- **19 total errors** across all documents

Copy this file to both your `data/` and `tests/` folders.

Troubleshooting

Docker images not visible in WSL

Your WSL is using a different Docker context. Fix with:

```
docker context use desktop-linux
```

Permission denied errors

Make sure the script is executable:

```
chmod +x apex-arena
```

Task validation fails

Run the validate command to see detailed errors:

```
./apex-arena validate-task schema_validator
```

Common issues:

- Missing `task.yaml` file
- `validation.json` in wrong location (should be in `tests/`)
- Dockerfile not using `FROM python:3.11-slim`

Container keeps failing

Debug by keeping the container running:

```
./apex-arena test-solution schema_validator --keep-container

# Then inspect it
docker exec -it apex_test_schema_validator_XXXXX bash
```

Platform Support

Platform	Status	Notes
Linux	Full support	Native
macOS	Full support	Requires Docker Desktop
Windows	Full support	Requires Docker Desktop + WSL2

Quick Reference

```
# List all tasks
./apex-arena list-tasks

# Validate a task structure
./apex-arena validate-task <task_name>

# Test a task
./apex-arena test-solution <task_name>

# Test with force rebuild
./apex-arena test-solution <task_name> --force

# Test with debug mode
./apex-arena test-solution <task_name> -v -k
```